Programação de Aplicações Web

Aula 4: Layouts Modernos e Responsividade

7. Layouts Modernos (Substituindo Floats e Tabelas)

Antigamente o design web dependia de adaptações problemáticas. A nova era do CSS foca nas soluções nativas **Flexbox** (Pense em 1 dimensão) e **CSS Grid** (Pense em 2 dimensões).

Flexbox (Flexible Box Layout)

Desenvolvido para organizar o espaço e alinhar os elementos dentro de um container, de forma "flexível".

- **Flex Container:** O elemento pai onde você declara `display: flex;`.
- **Flex Items:** Os filhos diretos, que são transformados em elementos maleáveis.

```
.container {
  display: flex; /* Transforma o comportamento tradicional */
  flex-direction: row; /* Define se fluem em linha ou coluna */
  justify-content: center; /* Alinhamento ao redor/no centro */
}
```

7. Propriedades do Flexbox

O poder real do display flex reside no alinhamento:

- **Direção do Fluxo (`flex-direction`):**
  - O padrão é `row` (linha, lado a lado). Pode ser `column` (empilhado, de cima para baixo).
- **Alinhamento do Eixo Principal (`justify-content`):**
  - Controla os espaços. Valores comuns: `start` , `center` , `end` , `space-between` , `space-evenly` .
- **Alinhamento do Eixo Transversal (`align-items`):**
  - Como comportar o centro. Ex: `center` para centralizar verticalmente.
- **Quebra de linha (`flex-wrap`):**
  - Se faltar espaço horizontal, os itens quebram para a próxima linha natural. `wrap` (permitir quebra), `nowrap` (padrão)
- **Preenchimento dos espacos (`gap`):**
  - Espaço entre as linhas de células. `gap: 20px;` (Não precisa colocar marigns em cada iten!)

7. CSS Grid Layout (Grelha/Grade)

Se Flexbox é incrível para uma linha ou uma coluna, o CSS Grid Layout é para o layout completo de uma página **em duas dimensões (Linhas e Colunas ao mesmo tempo)**.

- **Grid Container:** `display: grid;` .
- Planeja a estrutura em *tracks* (linhas ou colunas virtuais).

```
.layout {
  display: grid;
  /* Cria 3 colunas iguais. 1fr significa uma "fração do espaço livre" */
  grid-template-columns: 1fr 1fr 1fr;
  /* Cria 2 linhas virtuais */
  grid-template-rows: 100px auto auto;
  gap: 15px;
}
```

Desta forma criamos uma galeria de fotos perfeita sem "matemática de larguras percentuais" mirabolante que acontecia nos métodos antigos.

8. Web Responsiva (Design Adaptável)

A web não é apenas lida em monitores de Desktop, mas principalmente em telas compactas e com orientações varíaveis (celulares e tablets).

- O **Design Responsivo** é construir páginas que se adaptam e recompoem sua forma visual dependendo do dispositivo em que estão.
- Substituimos em muitos locais as larguras fixas por proporções elásticas (`width: 100%` , flexibilidade `max-width`).

Unidades Relativas

As unidades fundamentais da web responsiva.

- `%` : Relativo ao elemento pai.
- `vw` (Viewport Width): `100vw` é toda a largura do ecrã disponível.
- `vh` (Viewport Height): `100vh` é a atura total vertical da tela.
- `rem` : Relativa ou modificador o tamanho base (root) definido pela tag `<html>` .

8. Media Queries (@media)

São **perguntas/condições** criadas que o navegador tenta responder sobre o ambiente (resolução / tamanho de tela). Se a condição for verdadeira, entra numa nova dimensão de estilos (modificando, sobreescrevendo outras regras css). Desta forma conseguimos criar **Breakpoints** (Pontos de Quebra).

```
/* Essa é a folha normal para mobile e telas pequenas (Mobile First) */
.coluna { width: 100%; }

/* Regra disparada SOMENTE QUANDO a tela possuir MAIS que 768px */
@media screen and (min-width: 768px) {
  .coluna { width: 50%; }
}

@media screen and (min-width: 1024px) {
  /* Em telas gigantes passa a ser 3 colunas */
  .coluna { width: 33.33%; }
}
```

8. O Princípio Mobile First

É uma estratégia crucial que virou o novo padrão com as restrições e tráfego maior em celulares.

- Projete primeiro para o ambiente **móvel/menor/celular**. Este é o CSS *Padrão* Base, que fica livre de amarras na folha.
- Em seguida, estenda as regras com os limites via **Media Queries** `min-width` sempre que houver espaço extra para preencher (Desktop).

Vantagem do Mobile First: A maioria das experiências no Desktop requer de forma complexa o CSS em duas dimensões (Grid e colunas elaboradas em tela solta), onde Mobile é um modelo contínio (Uma torre central).

É muito mais simples escrever CSS de forma fluida a partir de uma "Caixa Vertical única" e dividí-la quanto "haver mais espaço dispnível de largura", do que criar o sistema com colunas gigantes complexas e de repente quebrar blocos forçadamente em "mobile view" tirando colunas flutuantes através de `max-width` e afins.